



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 30-10-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

- 1.- Separar del arco la parte del letrero y las puertas o rejas
- 2.-Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHC desplazándose las letras de derecha a izquierda como si fuera letrero LCD/LED de forma cíclica
- 3.- Hacer que al presionar una tecla una de las puertas se abra por medio de una rotación y la otra puerta se abra por medio de un deslizamiento.

Extra 5 puntos: Su dado de 8 caras cae girando sobre el piso, y muestra un número random siempre hacia arriba, se repite la tirada al presionar una tecla.

El código que obtenemos es el siguiente:

```
Texture brickTexture;  
Texture dirtTexture;  
Texture plainTexture;  
Texture pisoTexture;  
Texture AgaveTexture;
```

```
Texture Letrero;  
Texture Puerta;  
Texture dadoTexture;  
Texture Arco;
```

```
Skybox skybox;
```

```
Model Arco_M;  
Model Letrero_M;  
Model PuertaDere_M;  
Model PuertaIzq_M;  
Model Dado_M;
```

```
brickTexture = Texture("Textures/brick.png");  
brickTexture.LoadTextureA();  
dirtTexture = Texture("Textures/dirt.png");  
dirtTexture.LoadTextureA();  
plainTexture = Texture("Textures/plain.png");  
plainTexture.LoadTextureA();  
pisoTexture = Texture("Textures/piso.tga");  
pisoTexture.LoadTextureA();  
AgaveTexture = Texture("Textures/Agave.tga");  
AgaveTexture.LoadTextureA();  
Puerta = Texture("Textures/made.jpg");  
Puerta.LoadTextureA();  
dadoTexture = Texture("Textures/Dado.png");  
dadoTexture.LoadTextureA();  
  
Letrero = Texture("Textures/Letrero.png");  
Letrero.LoadTextureA();  
  
Arco_M = Model();  
Arco_M.LoadModel("Models/arco.obj");  
Letrero_M = Model();  
Letrero_M.LoadModel("Models/Letrero.fbx");  
PuertaDere_M = Model();  
PuertaDere_M.LoadModel("Models/PuertaDere.obj");  
PuertaIzq_M = Model();  
PuertaIzq_M.LoadModel("Models/PuertaIzq.obj");  
Dado_M = Model();  
Dado_M.LoadModel("Models/Dado.fbx");
```

```

// ---- Animación del Dado ----

// Variables de control de estado
bool dadoAnimando = false; // Indica si el dado está actualmente en movimiento o animado
bool teclaIPresionada = false; // Detecta si la tecla para iniciar la animación fue presionada
bool dadoEnSuelo = false; // Determina si el dado ya cayó al suelo y debe detenerse

// Variables físicas y de transformación
glm::vec3 posDado(5.0f, 10.0f, -5.0f); // Posición inicial del dado en el espacio 3D
glm::vec3 velDado(0.0f, 0.0f, 0.0f); // Velocidad inicial (sin movimiento al comenzar)
float gravedad = -9.8f; // Valor de la gravedad que afectará al dado
float rotacionDado = 0.0f; // Ángulo acumulado de rotación del dado
glm::vec3 rotDadoActual(0.0f); // Vector que almacena la rotación actual en los tres ejes
glm::vec3 rotVelocidad(180.0f, 240.0f, 200.0f); // Velocidad angular del dado en grados por segundo

// Control de la cara visible del dado
int caraDado = 0; // Guarda el número de la cara superior del dado (inicialmente 0)

// Bucle principal de renderizado: se ejecuta mientras la ventana esté abierta
while (!mainWindow.getShouldClose())
{
    // Calcula el tiempo transcurrido entre frames para animaciones suaves
    GLfloat now = glfwGetTime(); // Obtiene el tiempo actual del sistema
    deltaTime = now - lastTime; // Calcula la diferencia de tiempo entre cuadros
    deltaTime += (now - lastTime) / limitFPS; // Ajuste adicional para limitar los FPS
    lastTime = now; // Actualiza el último tiempo registrado

    // Incrementa la variable de ángulo para las animaciones dependientes del tiempo
    angulovaria += 0.5f * deltaTime; // Aumenta gradualmente el ángulo según el tiempo

    //Recibir eventos del usuario
    glfwPollEvents();
    camera.keyControl(mainWindow.getKeys(), deltaTime);
    camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());

    tiempoAcumulado += deltaTime;
    if (tiempoAcumulado >= intervaloCambio) {
        tiempoAcumulado = 0.0f;
        indiceInicio++;
        if (indiceInicio > (int)offsets.size() - letrasVisibles)
            indiceInicio = 0;
    }

    // Abrir/cerrar con tecla F
    if (mainWindow.getKeys()[GLFW_KEY_F]) {
        if (!teclaOPresionada) {
            abierta = !abierta;
            teclaOPresionada = true;
        }
    }
    else {
        teclaOPresionada = false;
    }

    // ---- Animación ----

    // Define el ángulo objetivo de la puerta según su estado (abierta o cerrada)
    float targetAng = abierta ? -90.0f : 0.0f;

    // Comprueba si el ángulo actual difiere del objetivo (si aún debe moverse)
    if (fabs(ang - targetAng) > 0.1f) {

        // Calcula el incremento del ángulo en función de la velocidad y el tiempo transcurrido
        float step = vel * deltaTime / 10;

        // Si el ángulo actual es menor al objetivo, aumenta gradualmente hasta alcanzarlo
        if (ang < targetAng)
            ang = std::min(ang + step, targetAng);
        // Si es mayor, disminuye hasta llegar al valor deseado
        else
            ang = std::max(ang - step, targetAng);
    }
}

```

```

// ---- Animación de la Puerta Derecha (abrir/cerrar con tecla G) ----
if (mainWindow.getKeys()[GLFW_KEY_G]) // Detecta si se presiona la tecla 'G'
{
    // Si la tecla no estaba presionada antes y la puerta no está animándose
    if (!teclaPPresionada && !puertaDerechaAnimando)
    {
        puertaDerechaAnimando = true; // Inicia la animación de apertura o cierre
        teclaPPresionada = true; // Marca que la tecla ya fue detectada

        // Determina la fase de la animación según el estado de la puerta
        if (!puertaDerechaAbierta)
        {
            fase = 1; // Fase de apertura
        }
        else
        {
            fase = 3; // Fase de cierre
        }

        // Reinicia el progreso de animación
        progresoAnim = 0.0f;
    }
}
else
{
    // Si la tecla no está siendo presionada, restablece el estado
    teclaPPresionada = false;
}

// ---- Lógica de movimiento de la puerta ----
if (puertaDerechaAnimando) // Si la puerta está en proceso de animación
{
    // Incrementa el progreso de animación de forma proporcional al tiempo
    progresoAnim += deltaTime / 20;

    // Calcula el parámetro de interpolación (va de 0.0 a 1.0)
    float t = progresoAnim / duracionFase;

// ---- Fase 1: Apertura inicial ----
if (fase == 1)
{
    // Mueve la puerta gradualmente hacia la posición de apertura (eje Z)
    puertaDerechaPos.z = glm::min(t, 1.0f) * 0.5f;

    // Si completó la fase, pasa a la siguiente
    if (t >= 1.0f)
    {
        fase = 2; // Avanza a la fase 2 (apertura completa)
        progresoAnim = 0.0f; // Reinicia el progreso
    }
}

// ---- Fase 2: Finaliza apertura ----
else if (fase == 2)
{
    // Interpola la puerta hasta el punto final de apertura
    puertaDerechaPos = glm::vec3(glm::min(t, 1.0f) * 4.0f, 0.0f, 1.0f);

    // Si terminó la animación de apertura
    if (t >= 1.0f)
    {
        puertaDerechaAnimando = false; // Detiene la animación
        puertaDerechaAbierta = true; // Marca la puerta como abierta
        fase = 0; // Reinicia las fases
    }
}

// ---- Fase 3: Inicio de cierre ----
else if (fase == 3)
{
    // Mueve la puerta en dirección contraria (cierre)
    puertaDerechaPos = glm::vec3((1.0f - glm::min(t, 1.0f)) * 4.0f, 0.0f, 1.0f);
}
}

```

```

        // Si la fase de cierre inicial termina
        if (t >= 1.0f)
        {
            fase = 4; // Cambia a la siguiente fase (cierre completo)
            progresoAnim = 0.0f; // Reinicia el contador
        }
    }

    // ---- Fase 4: Finaliza cierre ----
    else if (fase == 4)
    {
        // Ajusta la posición final para que regrese completamente a su punto inicial
        puertaDerechaPos.z = (1.0f - glm::min(t, 1.0f)) * 0.5f;

        // Si ya terminó de cerrarse
        if (t >= 1.0f)
        {
            puertaDerechaAnimando = false; // Detiene la animación
            puertaDerechaAbierta = false; // Marca la puerta como cerrada
            fase = 0; // Reinicia el ciclo
        }
    }
}

// ---- Animación del Dado ----
if (mainWindow.getKeys()[GLFW_KEY_L]) // Detecta si se presiona la tecla 'L'
{
    if (!teclaIPresionada) // Verifica que la tecla no estuviera ya presionada
    {
        teclaIPresionada = true; // Marca que se presionó la tecla

        // Si el dado no está animándose ni en el suelo, inicia la animación
        if (!dadoAnimando && !dadoEnSuelo)
        {
            dadoAnimando = true; // Activa la animación
            velDado = glm::vec3(0.0f, 6.0f, 0.0f); // Asigna velocidad inicial hacia arriba
            rotDadoActual = glm::vec3(0.0f); // Reinicia la rotación acumulada

            // Asigna velocidades de rotación aleatorias para cada eje (en grados)
            rotVelocidad = glm::vec3(
                180.0f + rand() % 90,
                180.0f + rand() % 90,
                180.0f + rand() % 90
            );

            // Selecciona aleatoriamente una cara del dado (0-8)
            caraDado = rand() % 9;
        }

        // Si el dado está en el suelo, se reinicia para volver a lanzar
        else if (dadoEnSuelo)
        {
            dadoEnSuelo = false; // Indica que ya no está en el suelo
            dadoAnimando = false; // Detiene la animación temporalmente
            posDado = glm::vec3(5.0f, 10.0f, -5.0f); // Restaura la posición inicial
            velDado = glm::vec3(0.0f); // Velocidad en cero
            rotDadoActual = glm::vec3(0.0f); // Reinicia rotación
        }
    }
}
else
{
    // Si la tecla ya no está presionada, resetea su estado
    teclaIPresionada = false;
}

// ---- Actualización del movimiento y rotación del dado ----
if (dadoAnimando)
{
    // Aplica la gravedad al dado en el eje Y
    velDado.y += gravedad * deltaTime;

    // Actualiza la posición del dado según su velocidad
    posDado += velDado * deltaTime;

    // Actualiza la rotación del dado en función de la velocidad angular
    rotDadoActual += rotVelocidad * deltaTime;
}

```

```

// Actualiza la rotación del dado en función de la velocidad angular
rotDadoActual += rotVelocidad * deltaTime;

// ---- Detección de contacto con el suelo ----
if (posDado.y <= 0.5f) // Si la posición Y es menor o igual al nivel del suelo
{
    posDado.y = 0.5f;          // Coloca el dado justo sobre el suelo
    dadoAnimando = false;      // Detiene la animación
    dadoEnSuelo = true;        // Marca que el dado ha caído
}
}

```

```

glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
model = glm::scale(model, glm::vec3(30.0f, 1.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
pisoTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[2]->RenderMesh(); // piso

```

```

/*
 * Arco con letrero
 */
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-10.0f, -1.5f, -2.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Arco.UseTexture();
Arco_M.RenderModel(); // modelo de arco
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Letrero_M.RenderModel(); // letrero blanco

glm::mat4 letraModel = glm::mat4(1.0f);
letraModel = glm::rotate(letraModel, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 2.0f, 2.0f));
Letrero.UseTexture();

for (int i = 0; i < letrasVisibles; i++) {
    int idx = (indiceInicio + i) % offsets.size();

    glm::vec2 toffset = offsets[idx];
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));

    glm::mat4 letraModel = glm::mat4(1.0f);
    letraModel = glm::rotate(letraModel, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    float startX = basePos.x + (letrasVisibles - 1) * separacion;
    letraModel = glm::translate(letraModel,
        glm::vec3(startX - i * separacion, basePos.y, basePos.z));

    letraModel = glm::scale(letraModel, escalaLetras);
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(letraModel));

    meshList[4]->RenderMesh();
}

```

```

//Puertas
model = modelaux;
model = glm::translate(model, glm::vec3(-0.2f, 0.0f, 0.0f));
model = glm::rotate(model, -ang * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Puerta.UseTexture();
PuertaIzq_M.RenderModel();

model = modelaux;

model = glm::translate(model, glm::vec3(puertaDerechaPos.x, puertaDerechaPos.z, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Puerta.UseTexture();
PuertaDere_M.RenderModel();

//dado
model = glm::mat4(1.0);
model = glm::translate(model, posDado);
model = glm::rotate(model, glm::radians(40.0f), glm::vec3(1, 0, 0));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0, 0, 1));

if (dadoAnimando) {
    model = glm::rotate(model, glm::radians(rotDadoActual.x), glm::vec3(1, 0, 0));
    model = glm::rotate(model, glm::radians(rotDadoActual.y), glm::vec3(0, 1, 0));
    model = glm::rotate(model, glm::radians(rotDadoActual.z), glm::vec3(0, 0, 1));
}
else if (dadoEnSuelo) {
    switch (caraDado) {
        case 0: break;
        case 1: model = glm::rotate(model, glm::radians(80.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 0, 1));
            break;
        case 2: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 1, 0)); break;
        case 3: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
        case 4: model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
        case 5: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 0, 1)); break;
        case 6: model = glm::rotate(model, glm::radians(360.0f), glm::vec3(0, 1, 0)); break;
        case 7: model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0, 1, 0)); break;
        case 8: model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
    }
}

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dadoTexture.UseTexture();

```

```

    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    dadoTexture.UseTexture();
    Dado_M.RenderModel();
    glDisable(GL_BLEND);
    glUseProgram(0);
    mainWindow.swapBuffers();
}

return 0;
}

```



Problemas:

Tuve demasiados problemas, entre ellos el modelo, yo modele la puerta que entregue en el previo pero modele una sola puerta, entonces tuve que descargar y hasta pague por un modelo pero tontamente, no leí bien los requisitos y fue otra de una sola puerta. Tuve que cambiar algo en blender y entonces el modelo y los pivotes quedaron totalmente volteados en la puerta derecha. después en el letrero, nunca le entendí bien al uso de GIMP y me salió un horror el letrero, la verdad ya no tuve tiempo de modificarlo y preferí entregarlo así.

Conclusión:

Para ser honesta, no tuve 100% la atención en la clase para poder hacer conclusiones o comentarios sobre este, los videos de verdad ayudan mucho, me sacan de muchas dudas y eso sigue siendo una muy buena idea. La práctica si se me hizo sumamente compleja pero más que nada por el hecho de los tiempos que manejo, el estrés ya estaba en su peor punto y el que no me salieron las cosas pues fue mucho peor.